Week 7 Lab: Submission

Name: 龍偉亮
Student ID: 314561005
Team number: 17
Model used: claude-haiku-5-5
Date: 2026-10-21

Part 1: Wire up the tools

1.1 Your `handle_tool_call` code

```
def handle_tool_call(tool_name: str, tool_input: dict):
    try:
        if tool_name == "calculator":
            result = run_calculator(tool_input["expression"])
            # Design decision: "Error: ..." means the call failed, so flag it.
            return result, result.startswith("Error:")
        if tool_name == "search_knowledge_base":
            return search_knowledge_base(tool_input["query"]), False
        if tool_name == "get_course_calendar":
            return get_course_calendar(tool_input["item"]), False
        return (f"Unknown tool '{tool_name}'. Available tools: calculator, "
                "search_knowledge_base, get_course_calendar."), True
    except Exception as e:
        return f"{type(e).__name__}: {e}", True
```

1.2 Design question

Should a calculator result that starts with `Error:` be reported with `is_error=True`? Your decision and reason (one sentence):

Yes, it's an error after all. The model should know the call failed and retry.

1.3 Baseline results

> Run `python week7_lab_starter.py --v0 --tests`. Record what actually happened.

Query	Tool called and arguments (or "no tool")	Final answer	As expected? (yes/no)
What is 247 times 38?	`calculator({"expression": "247 * 38"})`	247 times 38 is 9,386	yes
What is covered in Week 6?	`get_course_calendar({"item": "Week 6"})` → "No date found"; then `search_knowledge_base({"query": "Week 6 topics and content"})`	Week 6 is async and covers why prompts fail: hallucination, sycophancy, sensitivity, over-refusal, bias	no (wrong tool first, then recovered)
When is the Week 7 lab due?	`get_course_calendar({"item": "Week 7 lab due date"})`	Due Wednesday, October 21, 23:59	yes

Query	Tool called and arguments (or "no tool")	Final answer	As expected? (yes/no)
What is the capital of France?	no tool	The capital of France is Paris.	yes

If anything was not as expected, what do you think caused it?

The Week 6 content question went to `get_course_calendar` first, which returned "No date found". The model then fell back to `search_knowledge_base` and answered correctly. I think the V0 calendar description ("Finds course dates and information") is the cause: "and information" makes it sound like a general course lookup, so it overlaps with the knowledge base. This is a wrong-tool finding (mode 1) that cost one extra call.

Part 2: Diagnose the provided descriptions

> For each mode: write your **prediction first**, then run `python week7_lab_starter.py --v0 --modes --repeat 5`, then fill in the record. Counts matter more than a single run, because results vary.
> Outcome codes: **A** = failed on the starter queries; **B** = failed only under `--stress`; **C** = did not fail (also complete the "did not fail" report for that mode, below).

Failure mode 1: Wrong tool

- **My prediction** (query, and the V0 words I expected to cause it): "What are the deliverables for CP1?" will go to `get_course_calendar` in some runs. Its V0 text, "Finds course dates and information", overlaps with the knowledge base ("Searches course information"), and CP1 looks like a dated item. A second guess is "How many techniques are covered in Week 5?" going to `calculator`, because "How many" is numeric and the calculator V0 text says "Use for anything involving numbers".
- **Query used:** "How many techniques are covered in Week 5?" (also "What are the deliverables for CP1?" and "When is the Week 7 lab due?", plus the Week 6 content question in the baseline and "Week6" in mode 2)
- **Runs: 5. What the model did** (tool sequences and counts, for example "calculator x3, no tool x2"):
  - Week 5 techniques: `get_course_calendar -> search_knowledge_base` x5
  - CP1 deliverables: `search_knowledge_base` x5 (correct)
  - Week 7 lab due: `get_course_calendar` x5 (correct)
- **Tool errors** (n of 5), with an example if any: 0 of 5 flagged as errors. The calendar returned "No date found for that item", which is a normal result and not an error. So each Week 5 run wasted one call. Example: `get_course_calendar({"item": "Week 5"})`.
- **Correct answer; was the final answer correct?** Four techniques (zero-shot, few-shot, chain-of-thought, self-correction). The final answer was correct in all 5 runs, because the model recovered by calling the knowledge base.
- **Outcome:** A (failed on the starter queries: the wrong tool was chosen first in 5 of 5 runs, and the model recovered each time). My prediction was only partly right. I expected the CP1 query to go to the calendar, but it did not. The Week 5 query did go to the calendar, not to the calculator as my second guess said.
- **Which words in the V0 description made this plausible:** "Finds course dates **and information**" in `get_course_calendar`. It overlaps with the knowledge base, and "Week 5" looks like a schedule item. It also lacks any "do not use for content" boundary.

Failure mode 2: Wrong parameters

- **My prediction** (query, and the V0 words I expected to cause it): "What is the square root of 144 plus 15 percent of 240?" The calculator V0 text only says "Does calculations", and the `expression` parameter only says "The thing to calculate". Nothing says which syntax is allowed, so I expect the model to send `sqrt(144)`, `15%` or `^`, which a parser limited to numbers and + - * / ** % // will reject.

- **Query used:** "What is the square root of 144 plus 15 percent of 240?" (plus stress: "15% of 240 plus 3 squared, all divided by 2" and "2 to the power of 200")
- **Runs: 5. What the model did** (tool sequences and counts): `calculator -> calculator` x4, no tool x1. Stress: "15% of 240 plus 3 squared..." gave `calculator -> calculator` x3 and no tool x2; "2 to the power of 200" always used 2 to 4 calculator calls.
- **Arguments the model sent, and what the tool returned:** `calculator({"expression": "sqrt(144) + 0.15 * 240"})` returned `Error: only numbers and + - * / ** % // are allowed` (errors in 4 of 5 runs). In the stress queries it sent `((15/100) * 240 + 3^2) / 2`, and `2^200` was rejected in 5 of 5 runs. Both used `^`, which the parser does not support.
- **Did the model recover from an error? How?** Yes, in every case. It retried with a simplified expression (the answer 12 + 36 = 48 shows it replaced `sqrt(144)` with 12). For 2^200 it computed 2^100 and squared it, and still gave the right number.
- **Correct answer; was the final answer correct?** 48, and 22.5 for the stress query. Both were correct. Each failure cost extra calls but did not give a wrong answer.
- **Outcome:** A (the tool errored in 4 of 5 runs on the starter query).
- **Which words in the V0 description made this plausible:** "Does calculations. Use for anything involving numbers" and the parameter "The thing to calculate" give no supported syntax, so the model assumed `sqrt()`, `^` and percent notation would work. My prediction was right.

Failure mode 3: Tool called unnecessarily

- **My prediction** (query, and the V0 words I expected to cause it): "What is 10 divided by 2?" (a stress query) is the most likely unnecessary call, because the calculator V0 text says "Use for anything involving numbers". I expect "What is the capital of France?" and the chain-of-thought question to stay tool-free. The knowledge base V0 text says "Use only if you do not already know the answer", which discourages calls on general knowledge.
- **Query used:** the starter queries (hallucination, capital of France, chain-of-thought), the stress queries, and my own "What is 12 plus 30?" and "What is 7 times 8?" (full table in the extra report below)
- **Runs: 5. What the model did** (tool sequences and counts): the three starter queries had no tool in 15 of 15 runs. Stress: "Explain in one sentence why the sky is blue" and "What is 10 divided by 2?" had no tool in 10 of 10 runs. My own queries: "What is 12 plus 30?" `calculator` x5; "What is 7 times 8?" `calculator` x5; "What is 100 minus 1?" `calculator` x1, no tool x4.
- **Was the tool call worth it?** No. The model can do 12 + 30 and 7 x 8 reliably, so the call only added a round trip. The answer was still correct, so the cost is latency and tokens, not wrong answers.
- **Outcome:** C on the starter and stress queries (no failure in 25 runs), but it did fail on my own easy-arithmetic queries (details in the Mode 3 report below) (`calculator` 5 of 5 on 12 plus 30 and on 7 times 8). So it is an unnecessary-call failure that only shows up on simple sums. My prediction was partly wrong: I expected 10 divided by 2 to fail, and it did not.
- **Which words in the V0 description made this plausible:** "Use for anything involving numbers" in `calculator`. It has no lower bound on how simple the sum can be. The model follows it for small sums, but the starter and stress queries were easy enough that it skipped the tool.

Failure mode 4: Tool not called when it should be

- **My prediction** (query, and the V0 words I expected to cause it): "What does the code steward do?" will sometimes be answered from memory, because the knowledge base V0 text says "Use only if you do not already know the answer" and the model may think it knows what a steward is. "What is 48273 times 9167?" may also be answered from memory, with a wrong product, because "Does calculations" never says to use it for large numbers.
- **Query used:** "What does the code steward do?", "When is Assignment 1 due?", "What is 48273 times 9167?" (starter); "What is 987654 times 123456?", "Who is the code steward this week?", "What is the topic of Week 9?" (stress)
- **Runs: 5. What the model did** (tool sequences and counts): code steward `search_knowledge_base` x5; Assignment 1 `get_course_calendar` x5; 48273 x 9167 `calculator` x5 (3 single calls, 2 with a second call). Stress: 987654 x 123456 `calculator` x5; code steward this week and Week 9 both `get_course_calendar -> search_knowledge_base` x5.

- **Correct answer (from the knowledge base or by calculation), and the model's answer:** 442,518,591 and the model gave 442,518,591. For the code steward, the model matched the knowledge base text (PR reviewer for a period, at least twice, DECISIONS.md). For "this week" and "Week 9" the model said it could not find the answer, which is what the knowledge base supports.
- **Outcome:** C. No missed call in any starter or stress query, and the product was never answered from memory.
- **Which words in the V0 description made this plausible:** "Use only if you do not already know the answer" in `search_knowledge_base` could suppress calls, and "Does calculations" never says to use it for large numbers. My prediction was wrong: the model called the tool every time.

"Did not fail" report (complete one for each mode with outcome C)

> Skip this section if every mode failed. A complete report counts the same as a record of a failure. Delete the blocks you do not need.

Mode 3 (tool called unnecessarily): no failure on the starter and stress queries, but it failed on my own queries

A. Evidence. Queries, 5 runs each with `--v0 --repeat 5`:

Query	Written by	Condition (starter / stress / changed)	Counts (tool sequences over 5 runs)	Any failure?
What does the term 'hallucination' mean...?	starter	starter	no tool x5	no
What is the capital of France?	starter	starter	no tool x5	no
What is chain-of-thought prompting?	starter	starter	no tool x5	no
Explain in one sentence why the sky is blue.	starter	stress	no tool x5	no
What is 10 divided by 2?	starter	stress	no tool x5	no
What is 12 plus 30?	me	own	`calculator` x5	**yes**
How many days are in a week?	me	own	no tool x5	no
Name the three primary colors.	me	own	no tool x5	no

B. My deliberate attempts to break it. Predictions written before running:

My query	My prediction and reasoning	What happened (counts)
What is 7 times 8?	No tool. It is a single-digit times table fact, so I expected the model to answer directly.	`calculator` x5. My prediction was wrong.
What is 100 minus 1?	Mostly no tool, with a small chance of a calculator call because of "anything involving numbers".	no tool x4, `calculator` x1

C. Conditions I changed:

Condition	Query	Counts	Did it change anything?
Chinese	法國的首都是哪裡？	no tool x5	no
Irrelevant context	long complaint, then "what is 3 plus 4?"	no tool x5	no
Misspelled	wat is the capitol of Frnace	no tool x5	no

D. Reflection.

1. My verdict: no failure on the three starter and two stress queries or on the non-numeric and changed-condition queries (25 runs of starter and stress, plus 30 more), but it did fail on small arithmetic: `calculator` in 5 of 5 runs on "12 plus 30" and "7 times 8", and 1 of 5 on "100 minus 1". So it failed only on easy sums, which the starter and stress queries barely covered.
2. Why it held up elsewhere: general-knowledge questions have no numbers, so "Use for anything involving numbers" never applies. The knowledge base text "Use only if you do not already know the answer" also discourages calls, and a strong model knows the capital of France.
3. What would make it fail in a real project: more tools with broad "use for anything..." wording, a model that is less sure of itself, or users who ask many tiny calculations, where each extra call adds up in cost and latency.
4. What I would monitor: the fraction of runs where a tool was called on a set of "no tool needed" questions, including trivial sums. "It didn't fail today" on a handful of starter queries is not enough, because my own two extra queries failed where the starters did not.

Mode 4 (tool not called when it should be) did not fail

A. Evidence. Queries, 5 runs each with `--v0 --repeat 5`:

Query	Written by	Condition (starter / stress / changed)	Counts (tool sequences over 5 runs)	Any failure?
What does the code steward do?	starter	starter	`search_knowledge_base` x5	no
When is Assignment 1 due?	starter	starter	`get_course_calendar` x5	no
What is 48273 times 9167?	starter	starter	`calculator` x5, correct product	no
What is 987654 times 123456?	starter	stress	`calculator` x5	no
What is the deadline for CP1?	me	starter	`get_course_calendar` x5	no
What is 73 times 8191?	me	starter	`calculator` x5 (1 run had a tool error from "73 times 8191" first, then recovered), answer 597,943	no

B. My deliberate attempts to break it. Predictions written before running:

My query	My prediction and reasoning	What happened (counts)
How many deliverables does CP1 have?	The model might answer "five" from memory, since the count is a small number and sounds easy. The V0 text says to use the knowledge base only if it does not already know. I expected a knowledge base call, with a small risk of a miss.	`search_knowledge_base` x5, answered 5 correctly
Is Assignment 1 due before or after the Week 7 lab?	It needs two calendar lookups. I predicted two calendar calls, with a small risk of a guessed answer.	`get_course_calendar -> get_course_calendar` x5, correct

C. Conditions I changed:

Condition	Query	Counts	Did it change anything?
Chinese	code steward 是做什麼的？	`search_knowledge_base` x5, answer in Chinese and correct	no
Vague	wk 6 stuff	`get_course_calendar` -> `get_course_calendar` x5	partly: it asked the calendar for dates and offered to look up content, so it never searched the knowledge base for "week 6 stuff" (mode 1 behaviour, not a missed call)
Misspelled	cp 1 deadlin	`get_course_calendar` -> `get_course_calendar` x5, correct	no
Irrelevant context	long complaint, then "when is assignment 1 due?"	`get_course_calendar` x5, correct	no

D. Reflection.

1. My verdict: no clear missed call in 15 queries (75 runs) across 5 conditions: starter, stress, my own, two break attempts, and four changed conditions. The one soft spot was the vague "wk 6 stuff", which went to the calendar twice and never searched the knowledge base for content. That is closer to a wrong-tool failure than a missed call, since the model did call a tool.
2. Why I think it held up: each tool has a distinctive name, there are only three tools, and the queries in this course domain clearly point at a tool. A strong model also knows it cannot know course-specific facts such as Assignment 1's due date, so "Use only if you do not already know" did not suppress calls. The calculator's "Does calculations" was enough for big products.
3. What would make it fail in a real project: many more tools with similar names, long conversations, a smaller or updated model, or queries where the model really believes it knows the answer, such as a deadline that matches a typical course.
4. What I would monitor: a regression set of course-specific questions with known answers, run on every model or prompt change, checking that the expected tool was called and that the answer matches. "It didn't fail today" is a sample of a few dozen runs of one model version, and it says little about a future model or a rare phrasing.

Part 3: Fix it with my own descriptions

3.1 Final schemas

> Paste your final versions of the descriptions in `TOOLS`.

calculator: description

```
Evaluates one plain arithmetic expression exactly and returns the result as a number. Use it
for any multiplication, division or multi-step arithmetic where precision matters, and for date
differences (compute them from dates you got from get_course_calendar). Do NOT use it for
trivial arithmetic you can do reliably yourself, such as 2+2 or 10/2, and never for questions
about course content or dates. Supported: numbers, + - * / ** (power, exponent at most 100), %
(modulo), // (floor division), unary minus. NOT supported: sqrt(), percent signs (write 15% of
240 as 0.15 * 240), ^, commas in numbers, words, variables. Invalid input returns an 'Error:
...' message; fix the expression and retry.
```

calculator: parameter description (`expression`)

```
A single arithmetic expression using only digits, decimal points, parentheses and + - * / ** % //. Example: '(0.15 * 240 + 3 ** 2) / 2'. Use ** for powers, 12 ** 0.5 for a square root, no thousands separators.
```

search_knowledge_base: description

```
Looks up course CONTENT: what each week covers, what is in an assignment or checkpoint deliverable (e.g. CP1), and what a role such as the code steward does. Use it whenever the question is about this course's specific material, even if you think you know the answer, because course details may differ from general knowledge. Do NOT use it for deadlines or due dates (use get_course_calendar), for arithmetic (use calculator), or for general-knowledge questions unrelated to the course, such as geography or what chain-of-thought prompting is in general. Returns a short text passage, or 'No relevant information found'. It only knows weeks 5-7, assignment 1, CP1 and the code steward; if it finds nothing, say so instead of guessing.
```

search_knowledge_base: parameter description (`query`)

```
Short keyword phrase naming the topic, with spaces and normal spelling, e.g. 'week 6', 'cp1 deliverables', 'code steward', 'assignment 1'.
```

get_course_calendar: description

```
Looks up course DATES only: when something is due or was due. Use it for any 'when' or 'deadline' question about CP1, the Week 6 lab, the Week 7 lab or Assignment 1. Do NOT use it for what a week covers, what a deliverable contains or what a role does (use search_knowledge_base), and do not use it for arithmetic. Returns one sentence with the due date and time, or 'No date found' plus the list of items that have dates. For questions that compare dates, call it once per item, then use calculator for the difference.
```

get_course_calendar: parameter description (`item`)

```
Name of the dated item, lowercase, e.g. 'cp1', 'week 6 lab', 'week 7 lab', 'assignment 1'. Name only the item, not the question.
```

3.2 Before and after

> Run on my own descriptions with `--modes --repeat 5`, `--stress --repeat 5` and my extra queries (5 runs each).

Query	V0 result (5 runs)	My result (5 runs)	Better / same / worse
How many techniques are covered in Week 5? (mode 1)	`calendar -> KB` x5 (wrong tool first)	`KB` x5	better
What are the deliverables for CP1? (mode 1)	`KB` x5	`KB` x5	same
When is the Week 7 lab due? (mode 1)	`calendar` x5	`calendar` x5	same
What is covered in Week6? (mode 2)	`calendar -> KB` x5	`KB` x5	better
Square root of 144 plus 15 percent of 240 (mode 2)	`calc -> calc` x4, no tool x1; tool errors 4 of 5	no tool x5; tool errors 0 of 5; answer 48	better (but it skipped the calculator)
15% of 240 plus 3 squared, all divided by 2 (mode 2	`calc -> calc` x3, no tool x2; errors 3 of 5	`calc` x4, no tool x1; errors 0 of 5	better

Query	V0 result (5 runs)	My result (5 runs)	Better / same / worse
stress)	(`^`)		
2 to the power of 200 (mode 2 stress)	`calc` x3-4 per run; errors 5 of 5	`calc -> calc` x4, `calc` x3 x1; errors 5 of 5 (`2 ** 200` -> "exponent too large")	same (still fails, still recovers)
12 plus 30 (mode 3, mine)	`calc` x5	no tool x5	better
7 times 8 (mode 3, mine)	`calc` x5	no tool x5	better
What is 10 divided by 2? (mode 3 stress)	no tool x5	no tool x5	same
What is 48273 times 9167? (mode 4)	`calc` x5, correct	`calc` x5, correct	same
Who is the code steward this week? (mode 4 stress)	`calendar -> KB` x5	`KB` x5	better (one call fewer)
What is the topic of Week 9? (mode 4 stress)	`calendar -> KB` x5	`KB` x5	better
Days between the CP1 and Assignment 1 deadlines (mode 1 stress)	`calendar -> calendar` x5	`calendar -> calendar` x3, `+ calculator` x2	same answer (18), slightly better method

3.3 Re-test list

> Queries re-run with my final schemas (`results/mine_modes.txt` , `mine_stress.txt` , `mine_extra.txt`).

#	Query	Expected behaviour	Result with my final schemas (ok / not ok)	Notes
1	What is the capital of France? / 法國的首都是哪裡？	no tool	ok (5 of 5 each)	the misspelled version also had no tool
2	What is 12 plus 30? / What is 7 times 8? / 100 minus 1	no tool, easy sum	ok (5 of 5 each)	this fixes the mode 3 failure from V0
3	What is 73 times 8191? / 987654 times 123456? / 48273 times 9167	calculator, correct product	ok (`calculator` x5 each, correct)	the "skip trivial sums" rule did not stop big products from using the tool
4	What does the code steward do? / code steward 是做什麼的？	knowledge base	ok (`KB` x5 each)	the mode 3 fix did not cause a missed call
5	When is Assignment 1 due? / cp 1 deadlin / with long irrelevant context	calendar	ok (`calendar` x5 each)	misspelling and filler did not matter
6	wk 6 stuff	knowledge base for content	mostly ok (`KB -> calendar` x5)	now searches the knowledge base first, which V0 never did, then also fetches the lab date
7	2 to the power of 200	calculator or a clear message	not ok	the model still tried `2 ** 200` and hit "exponent too large" in 5 of 5 runs, then recovered by splitting the power

3.4 One fix explained

The exact words I changed (before -> after), and why I expected them to help:

`get_course_calendar` : "Finds course dates and information." -> "Looks up course DATES only: when something is due or was due... Do NOT use it for what a week covers, what a deliverable contains or what a role does (use search_knowledge_base)". I expected removing "and information" to take the calendar out of content questions, and the explicit "use search_knowledge_base" to say where those questions go. Result: the Week 5 and "Week6" queries changed from `calendar -> KB` x5 to `KB` x5.

Did the fix for one mode affect another? (especially modes 3 and 4)

Not in a bad way. The new calculator text ("Do NOT use it for trivial arithmetic... such as 2+2 or 10/2") made 12 plus 30 and 7 times 8 go from `calculator` x5 to no tool x5 (mode 3). Large products still used the calculator in 5 of 5 runs, with correct answers (mode 4). The same text also let the model answer the sqrt-plus-percent query without the tool (0 errors, answer 48), which is correct here but means a harder expression could be done in the model's head.

Anything that still fails:

"What is 2 to the power of 200?" still errors in 5 of 5 runs. The calculator text says the exponent is at most 100, and the model still sent `2 ** 200`. It then recovers by splitting the power, so the answer is right, but the description did not stop the bad call. I would add a sentence such as "for results beyond exponent 100, split the power yourself" or move the limit into the parameter description.

Part 4: Written responses

Question 1 (3–5 sentences)

Which failure mode was hardest to fix or to provoke, and why? What does this tell you about designing tool descriptions, and about how much evidence you need before you trust an AI component in your project?

Mode 4 (missed call) was the hardest to provoke: it did not fail in 15 queries, including Chinese, misspelled and vague versions, even though the V0 text says "Use only if you do not already know the answer". Mode 3 was almost as hard. It stayed hidden on the starter and stress queries, and only appeared when I tried easy sums like 12 plus 30, where the calculator was called in 5 of 5 runs. That tells me the weak wording ("anything involving numbers") was a real defect, but the starter queries were too easy to expose it, so a failure only shows up with the right query. For my project, one clean run, or even one clean query set, is weak evidence. I would want counts over many runs, deliberately awkward queries and a written record of the queries I tried before I trust a tool-using component.

Question 2 (2–3 sentences)

Imagine your team's AI component could use a tool that **changes something** (sends a message, creates an issue, edits a file, deletes data). Name that tool. Who or what should approve it before it runs, and what would happen if the model called it wrongly?

In my team's project the risky tool would be one that creates or edits a GitHub issue or pull request (for example `create_issue` or `merge_pull_request`). The model should only propose the action. A team member (the person who asked, or the code steward for merges) should approve it in the interface before it runs, with the exact title, body and target shown. If the model called it wrongly, we could get a duplicate or misleading issue, or an unreviewed merge into the main branch that is hard to undo, so merges should always need a human and issue creation should at least show a preview.

Optional: screenshots

> Add a terminal screenshot of anything you want to show. Check that your API key is **not** visible.

Final checklist

- [x] ~~Name, student ID and model are filled in at the top~~
- [x] ~~`handle_tool_call` code, `is_error` reasoning and the four baseline results are included~~
- [x] ~~All **four** modes have a prediction and a record with run counts~~
- [x] ~~Any mode with outcome C has a complete "did not fail" report~~
- [x] ~~Final schemas for **all three** tools, a before/after table and a re-test list are included~~
- [x] ~~Question 1 and Question 2 are answered~~
- [x] ~~My answers are specific to what I saw~~
- [x] ~~No API key appears anywhere in the document~~
- [x] ~~Saved as one PDF named `Week7_Lab_<student ID>.pdf`~~